

ArUco Markers Lab

Lab 1 to Lab 5 Report

Course: Hands-on Perception

Date: April 4, 2025

Pravin Oli

u1999699, pravin.oli.08@gmail.com

Gebreslassie Gebrecherkos Gebrezgabhier

u1999542, chereg2016@gmail.com

Abstract

This report provides a step by step implementation of Labs 1 to 5, which collectively explore the development of an ArUco marker-based augmented reality system using OpenCV in C ++. Starting from basic image processing, the labs progress through marker generation, detection, camera calibration, and culminate in real-time pose estimation and 3D object rendering. This report outlines implementation steps, usage instructions, parameters, and visual outputs, highlighting key technical insights and learning outcomes at each stage.

Contents

1	Introduction	3
2	General Objectives	3
3	Implementation	3
3.1	Lab 1: Fiducial Markers and Introduction to OpenCV	3
3.1.1	Description	3
3.1.2	Implementation Steps	4
3.1.3	How to Run	5
3.1.4	Data and Parameters	5
3.2	Lab 2: Generation of ArUco Markers with OpenCV	6
3.2.1	Description	6
3.2.2	Implementation Steps	7
3.2.3	How to Run	7
3.2.4	Data and Parameters	8
3.3	Lab 3: Detection of ArUco Markers with OpenCV	9
3.3.1	Description	9
3.3.2	Implementation Steps	9
3.3.3	How to Run	10
3.3.4	Parameters and Notes	10
3.4	Lab 4: Camera Calibration using ArUco Boards and OpenCV	10
3.4.1	Description	10

3.4.2	Implementation Steps	11
3.4.3	How to Run	12
3.4.4	Parameters and Data	12
3.5	Lab 5: Augmented Reality using ArUco Markers and OpenCV	13
3.5.1	Description	13
3.5.2	Implementation Steps	13
3.5.3	How to Run	14
3.5.4	Results and Discussion	14
4	Conclusion	15

1 Introduction

This report contains five ArUco labs in computer vision, focused on building an augmented reality system using ArUco markers and OpenCV in C++ . Each lab builds upon the previous one, introducing key concepts in computer vision, camera geometry, and real-time processing. Together, these labs progressively built not just a toolkit of C++ and OpenCV techniques, but also a conceptual understanding of how digital vision systems perceive and interact with the physical world.

- **Lab 1** introduced the development environment and basic video processing using OpenCV, including grayscale conversion, blurring, edge detection, and binarization.
- **Lab 2** focused on generating individual ArUco markers and marker boards using OpenCV's built-in dictionaries.
- **Lab 3** implemented real-time detection of ArUco markers from live video, overlaying marker contours, corners, and IDs.
- **Lab 4** applied marker detection for camera calibration, estimating intrinsic parameters to relate 3D world coordinates to 2D image points.
- **Lab 5** integrated all previous steps to build a simple augmented reality system, overlaying 3D virtual objects on live video using marker pose estimation.

2 General Objectives

- Set up a computer vision development environment using OpenCV and C++.
- Learn to generate and interpret ArUco markers and marker boards.
- Implement real-time marker detection and tracking from live video streams.
- Perform camera calibration using ArUco boards to estimate intrinsic parameters.
- Apply marker-based pose estimation to develop a simple augmented reality system.

3 Implementation

3.1 Lab 1: Fiducial Markers and Introduction to OpenCV

3.1.1 Description

The objective of Lab 1 was to set up a C++ development environment using OpenCV and CMake and implement a basic real-time video processing code using a webcam. In this lab we work with fundamental image processing operations such as grayscale conversion, Gaussian blurring, edge detection, and binarization. This applied several foundational image processing techniques such as:

- **Grayscale Conversion:** Converts color images into shades of gray. This reduces complexity by collapsing RGB values into a single intensity channel using `cv::cvtColor`.
- **Gaussian Blur:** Used to smooth the image and reduce noise before edge detection. It applies a Gaussian function over a kernel. This is helpful in reducing false edges.

- **Canny Edge Detection:** A multi-stage algorithm to detect a wide range of edges. It uses gradients and hysteresis thresholding for accuracy.
- **Simple Binarization:** Converts grayscale images into black-and-white using a fixed threshold, enhancing feature separation.
- **Adaptive Binarization:** Applies thresholding dynamically across image regions, making it robust to varying lighting conditions.

3.1.2 Implementation Steps

For implementation of this lab1, below is a detailed description we went through, with code snippets highlighting each processing stage:

- Installed required dependencies including OpenCV and CMake.
- A C++ program to:
 - Capture live video using `cv::VideoCapture`.
 - Convert frames to grayscale using `cv::cvtColor`.
 - Apply Gaussian blur and Canny edge detection.
 - Display original and processed video in separate OpenCV windows.
- Extended the program to include:
 - Simple binary thresholding using `cv::threshold`.
 - Adaptive thresholding using `cv::adaptiveThreshold`.
 - Interactive control via trackbars to adjust Canny thresholds in real time.
- Grayscale Conversion: This transforms each RGB frame into a grayscale image for simpler analysis.

Listing 1: Grayscale conversion

```
1 cv::cvtColor(imgOriginal, imgGrayScale, cv::COLOR_BGR2GRAY);
```

- Gaussian Blur: A Gaussian kernel of size 5×5 with a sigma of 1.8 is applied to suppress noise.

Listing 2: Gaussian blur with 5x5 kernel

```
1 cv::GaussianBlur(imgGrayScale, imgBlurred, cv::Size(5, 5), 1.8);
```

- Canny Edge Detection: Detects edges using the thresholds `lowTh = 45, highTh = 90`.

Listing 3: Canny edge detection

```
1 cv::Canny(imgBlurred, imgCanny, lowTh, highTh);
```

- Simple Binarization: All pixel values above 100 are set to 255 (white), others to 0 (black).

Listing 4: Simple binary thresholding

```
1 cv::threshold(imgGrayScale, imgBinarized, 100, 255, cv::  
    THRESH_BINARY);
```

- **Adaptive Binarization:** Applies dynamic local thresholding using a block size of 11 and a constant of 2.

Listing 5: Adaptive thresholding

```
1 cv::adaptiveThreshold(imgGrayScale, imgAdaptiveBinarized, 255,  
2   cv::ADAPTIVE_THRESH_MEAN_C, cv::THRESH_BINARY, 11, 2);
```

3.1.3 How to Run

Compilation: We need to create a directory named Build and then make it using cmake.

Listing 6: Build using CMake

```
1 mkdir build  
2 cd build  
3 cmake ..  
4 make
```

Execution: Here, 0 selects the default webcam (usually the laptop's integrated camera).

Listing 7: Run executable with webcam input

```
1 ./example 0
```

Output: This command opens four OpenCV windows showing:

- Original webcam video
- Simple global binarization output
- Adaptive binarization output
- Canny edge-detected output

Dependencies:

- OpenCV
- C ++ compiler
- CMake
- Functional webcam

3.1.4 Data and Parameters

- **Input:** Live video feed from webcam
- **Gaussian Blur:**
 - Kernel size: (5, 5)
 - Sigma: 1.8
- **Canny Edge Detection:**

- LowTh = 45 (adjustable)
- HighTh = 90 (adjustable)

- **Thresholding:**

- Simple: Threshold = 100
- Adaptive: Block size = 11, Constant = 2

Video Screenshots:

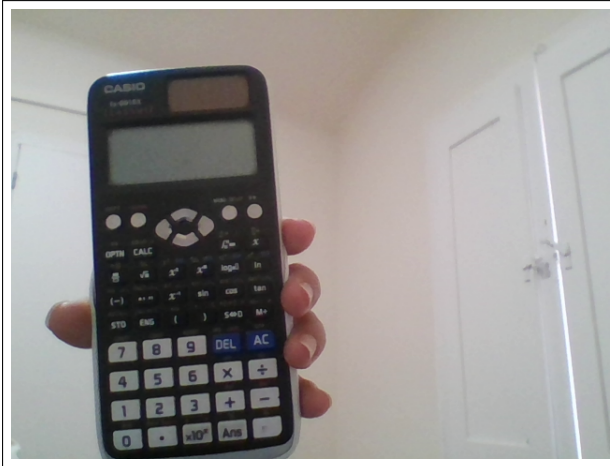


Figure 1: Original webcam image (live feed)

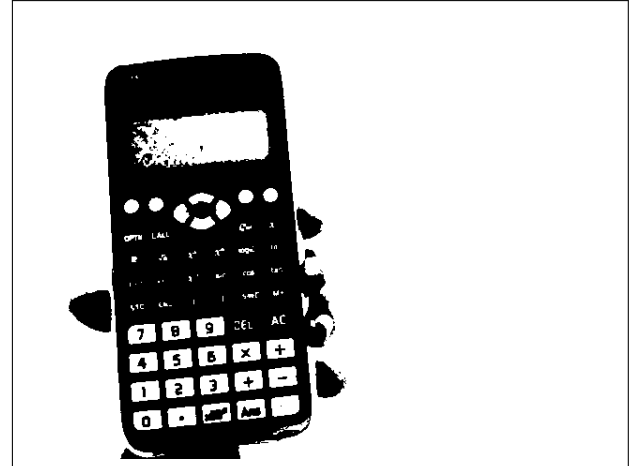


Figure 2: Simple global binarization

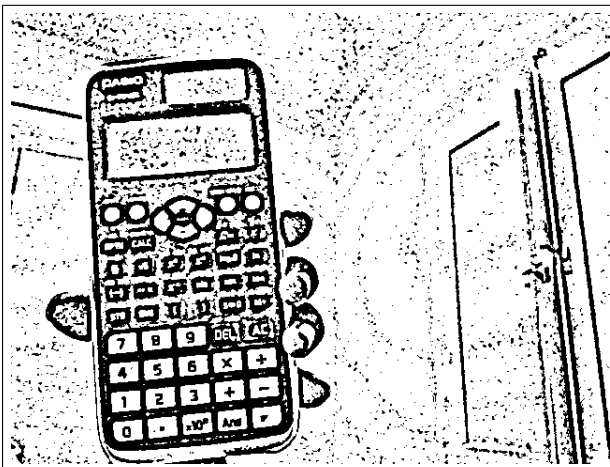


Figure 3: Adaptive binarization

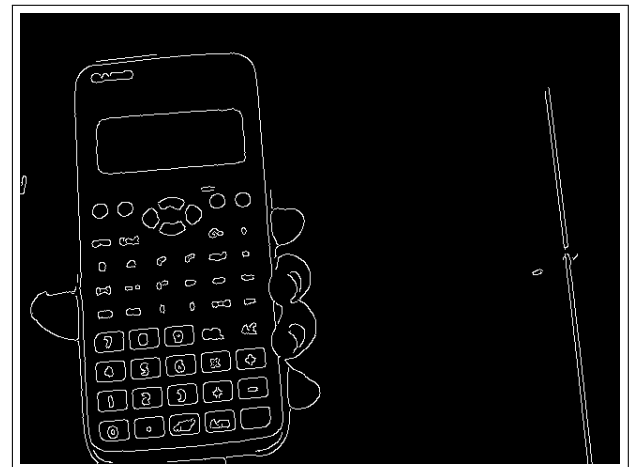


Figure 4: Canny edge detection

3.2 Lab 2: Generation of ArUco Markers with OpenCV

3.2.1 Description

The goal of Lab 2 was to develop two C++ command-line programs using OpenCV that generate fiducial markers known as ArUco markers. These markers are commonly used for pose estimation and calibration tasks. This lab focuses on understanding how to use predefined marker dictionaries and generate single markers as well as marker grids for camera-based detection.

We created two distinct tools:

- **generate_marker:** Generates a single ArUco marker with a white border.
- **generate_board:** Generates a board (grid) of multiple ArUco markers with specified size and spacing.

3.2.2 Implementation Steps

For implementation of this lab2, below is a detailed description we went through, with code snippets step by step:

- Two C++ programs that:
 - Use dictionaries such as `DICT_6X6_250`, `DICT_4X4_50`, or `DICT_ARUCO_ORIGINAL`.
 - Generate marker images and save them as PNG files.
- Added white borders to individual markers for better detection contrast.
- Used `cv::aruco::GridBoard` to assemble and export complete marker boards.
- Generate Single Marker: The marker is rendered using a predefined dictionary and an ID, surrounded by a white frame.

Listing 8: Generating an ArUco marker

```
1 cv::aruco::generateImageMarker(dictionary, markerId, markerSize,
   markerImage);
2 markerImage.copyTo(markerWithBorder(Rect(borderSize, borderSize,
   markerSize, markerSize)));
3 imwrite(filename, markerWithBorder);
```

- Generate Board of Markers: Creates a matrix of markers using `GridBoard` with user-defined marker size and spacing.

Listing 9: Generating an ArUco board

```
1 Ptr<cv::aruco::GridBoard> board = makePtr<cv::aruco::GridBoard>(
2     Size(cols, rows), float(markerSize), float(separation),
   dictionary);
3 drawPlanarBoard(board, boardImage.size(), boardImage, 10, 1);
4 imwrite(filename, boardImage);
```

3.2.3 How to Run

Compilation: Same as lab 1

Execution: Command-line arguments for executing.

- **generate_marker:**

Listing 10: Generate a single ArUco marker

```
1 ./generate_marker DICT_ARUCO_ORIGINAL 25 marker25.png
```

- **generate_board:**

Listing 11: Generate an ArUco board

```
1 ./generate_board DICT_ARUCO_ORIGINAL 2 4 0.043 0.0173 board.png
```

Output:

- `marker.png`: A square ArUco marker image (ID 25 shown).
- `board.png`: A grid with 2 rows and 4 columns of different markers.

3.2.4 Data and Parameters

- **Dictionaries:**
 - `DICT_6X6_250`
 - `DICT_4X4_50`
 - `DICT_ARUCO_ORIGINAL`
- **generate_marker Parameters:**
 - Dictionary Name (string)
 - Marker ID (integer)
 - Marker Size (pixels)
 - Output Filename (.png)
- **generate_board Parameters:**
 - Number of Rows
 - Number of Columns
 - Dictionary Name
 - Marker Size (pixels)
 - Marker Separation (pixels)
 - Output Filename (.png)

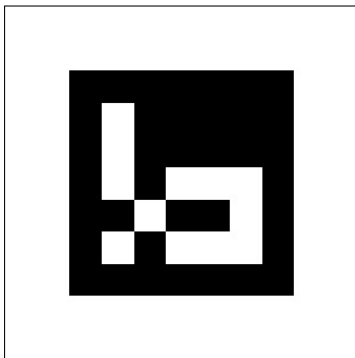
Marker Screenshots:

Figure 5: Single ArUco marker (ID 25)

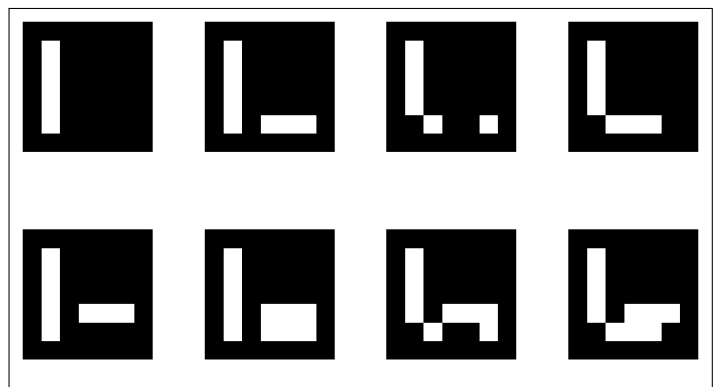


Figure 6: Grid of 8 ArUco markers

3.3 Lab 3: Detection of ArUco Markers with OpenCV

3.3.1 Description

The objective of Lab 3 was to implement a C++ program using OpenCV that detects ArUco markers in real-time from a webcam feed. The program identifies markers belonging to a user-specified dictionary and visually overlays detection results, including marker boundaries, marker IDs, and the top-left corner of each marker. This lab builds on the marker generation work from Lab 2 and introduces detection and visualization techniques using OpenCV's ArUco module.

3.3.2 Implementation Steps

For this lab3, here is the step-by-step breakdown of implementation process, we went while doing lab.

- Used OpenCV's `aruco::detectMarkers()` to perform real-time marker detection.
- Initialized the dictionary using `getPredefinedDictionary()` based on a user-specified string.
- Captured video using `cv::VideoCapture`.
- Detected markers, filtered rejected candidates, and drew detection overlays.
- Added visual cues:
 - Green borders around detected markers.
 - Marker ID displayed using `cv::putText`.
 - Red dot on the top-left corner using `cv::circle`.
- Dictionary Initialization: Maps input string to OpenCV's internal dictionary enumeration.

Listing 12: Selecting dictionary from command-line input

```
1 if (dictName == "DICT_6X6_250") dictEnum = cv::aruco::  
    DICT_6X6_250;  
2 else if (dictName == "DICT_4X4_50") dictEnum = cv::aruco::  
    DICT_4X4_50;  
3 else if (dictName == "DICT_ARUCO_ORIGINAL") dictEnum = cv::aruco  
    ::DICT_ARUCO_ORIGINAL;
```

- Marker Detection and Drawing: Detects all markers using the given dictionary and overlays the results.

Listing 13: ArUco marker detection and overlay drawing

```
1 cv::aruco::detectMarkers(frame, dictionary, markerCorners,  
    markerIds, parameters, rejectedCandidates);  
2 cv::aruco::drawDetectedMarkers(frame, markerCorners, markerIds);
```

- Marker ID and Corner Highlighting: Each detected marker shows its ID and a red dot at the top-left corner.

Listing 14: Drawing marker ID and red corner

```

1 putText(frame, "ID:␣" + to_string(markerIds[i]), markerCorners[i]
  ][0], FONT_HERSHEY_SIMPLEX, 0.8, Scalar(0, 255, 0), 2);
2 circle(frame, markerCorners[i][0], 5, Scalar(0, 0, 255), -1);

```

3.3.3 How to Run

Compilation: Same as previous lab.

Execution: command

Listing 15: Run detection with selected dictionary

```

1 ./detect_aruco DICT_ARUCO_ORIGINAL

```

Output:

- Live video feed with real-time ArUco detection.
- Markers framed in green with red dot at top-left corner.
- IDs displayed on top of each marker.

3.3.4 Parameters and Notes

- **Input:** Dictionary type provided as command-line argument
- **Default video source:** 0 (usually built-in webcam)
- **Detection Parameters:** Used default `DetectorParameters` object

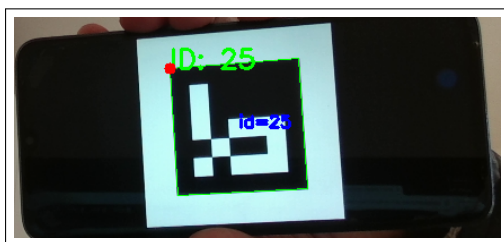


Figure 7: Single ArUco marker live detection

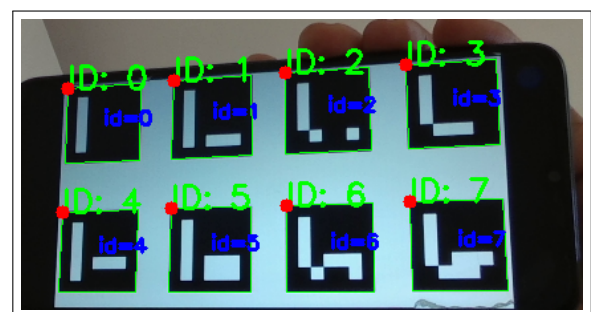


Figure 8: ArUco board live detection

3.4 Lab 4: Camera Calibration using ArUco Boards and OpenCV

3.4.1 Description

The purpose of Lab 4 was to perform camera calibration using a board of ArUco markers and OpenCV. Camera calibration allows to understand the transformation between the 3D world and its 2D image projection. This lab builds on the detection pipeline from Lab 3 and enables capturing calibration images that are later used to compute the camera's intrinsic parameters and distortion coefficients.

3.4.2 Implementation Steps

The calibration process included the following key stages, each supported by OpenCV's aruco module:

- Captured live video using `cv::VideoCapture`.
- Detected ArUco markers in real-time using the same detection pipeline from Lab 3.
- On pressing the `c` key, captured an image frame with a fully visible ArUco board.
- Repeated the process to collect 20–30 images of the same board at varied poses (angle, distance, tilt).
- After pressing the `ESC` key, performed camera calibration using the saved images.
- Saved the resulting camera parameters (intrinsic matrix and distortion coefficients) to a YAML file.
- **Dictionary and board setup:**

Listing 16: Create ArUco dictionary and board

```
1 Ptr<aruco::Dictionary> dictionary = getPredefinedDictionary(
    aruco::DICT_6X6_250);
2 Ptr<aruco::GridBoard> board = makePtr<aruco::GridBoard>(Size(
    cols, rows), markerLength, markerSeparation, *dictionary);
```

- **Load detector parameters:**

Listing 17: Read detection parameters from .yaml

```
1 FileStorage fs(detectorParamsFile, FileStorage::READ);
2 fs["adaptiveThreshWinSizeMin"] >> detectorParams->
    adaptiveThreshWinSizeMin;
3 fs["adaptiveThreshWinSizeMax"] >> detectorParams->
    adaptiveThreshWinSizeMax;
4 fs.release();
```

- **Image acquisition and marker detection:**

Listing 18: Capture and detect ArUco markers

```
1 cap >> frame;
2 aruco::detectMarkers(frame, dictionary, corners, ids,
    detectorParams, rejected);
3 aruco::drawDetectedMarkers(frame, corners, ids);
```

- **User-controlled image capture:**

Listing 19: Capture calibration image on 'c' press

```
1 if (key == 'c' && !ids.empty()) {
2     allCorners.push_back(corners);
3     allIds.push_back(ids);
4     markerCounterPerFrame.push_back((int)ids.size());
5 }
```

- **Camera calibration:**

Listing 20: Run calibration and save result

```

1 double repError = aruco::calibrateCameraAruco(
2     flatCorners, flatIds, markerCounterPerFrame,
3     board, imageSize, cameraMatrix, distCoeffs, rvecs, tvecs);
4
5 FileStorage fsOut(outputFile, FileStorage::WRITE);
6 fsOut << "cameraMatrix" << cameraMatrix;
7 fsOut << "distCoeffs" << distCoeffs;

```

3.4.3 How to Run

Compilation: compilation process is similar to previous labs.

Execution: The program expects 7 arguments:

Listing 21: Run camera calibration

```

1 ./calibrate_camera DICT_6X6_250 detector_params.yaml 4 5 0.04 0.01
   output.yaml

```

3.4.4 Parameters and Data

- **Input:**

- Dictionary name (e.g., DICT_6X6_250)
- Detector parameters YAML file
- Board dimensions (rows, cols)
- Marker side length and separation in meters
- Output filename (.yaml) to store calibration data

- **Output:** The output of this lab is a .yaml file containing the following calibration results:

- **cameraMatrix** — a 3×3 intrinsic matrix representing focal lengths and principal point.
 - **distCoeffs** — a vector of distortion coefficients modeling lens distortion.
 - **reprojection error** — a scalar indicating the overall accuracy of the calibration
- 4.26 reprojection Error.**

An example output is shown below:

Listing 22: Sample output calibration file in YAML format

```

1 cameraMatrix: !!opencv-matrix
2   rows: 3
3   cols: 3
4   dt: d
5   data: [ 599.89658574477437, 0., 348.03627756361237,
6           0., 645.2077100245109, 267.88954915256977,
7           0., 0., 1. ]

```

```

8
9 distCoeffs: !!opencv-matrix
10   rows: 1
11   cols: 5
12   dt: d
13   data: [ 0.076401770058834362, -0.057966032323305661,
14           0.0058361914472025468, 0.019741722120969724,
15           -0.41332022977089394 ]
16 Reprojection Error: Achieved reprojection error is 4.26 after
    accurate measurement.

```

3.5 Lab 5: Augmented Reality using ArUco Markers and OpenCV

3.5.1 Description

The aim of Lab 5 was to implement a basic augmented reality using fiducial ArUco markers. This was achieved by estimating the marker's 3D pose using previously computed camera calibration parameters, and then augmenting the real-world scene by overlaying either a coordinate frame or a synthetic cube on top of the detected marker.

Two programs were developed for this purpose:

- **pose_estimation:** Estimates the pose of a detected ArUco marker and overlays a live 3D coordinate axis on it.
- **draw_cube:** Uses the same pose estimation process to render a virtual cube that appears fixed on top of the marker in real time.

3.5.2 Implementation Steps

Pose Estimation

- Load camera intrinsic parameters from a calibration file:

Listing 23: Reading camera calibration parameters

```

1 FileStorage fs("camera_calib.yaml", FileStorage::READ);
2 fs["cameraMatrix"] >> cameraMatrix;
3 fs["distCoeffs"] >> distCoeffs;

```

- Detect the marker and estimate its pose:

Listing 24: Detecting and estimating marker pose

```

1 aruco::detectMarkers(frame, dictionary, corners, ids, parameters
2 );
3 aruco::estimatePoseSingleMarkers(corners, markerLength,
4 cameraMatrix, distCoeffs, rvecs, tvecs);

```

- Overlay the 3D axes and marker ID:

Listing 25: Drawing axis and ID overlay

```

1 drawFrameAxes(frame, cameraMatrix, distCoeffs, rvecs[i], tvecs[i]
2 ], 0.1f);
3 putText(frame, ss.str(), corners[i][0] + Point2f(10, -10),
4 FONT_HERSHEY_SIMPLEX, 0.5, Scalar(255,255,255), 1);

```

3D Cube Rendering

- same as pose estimation.
- Defined 8 cube corners in 3D space:

Listing 26: Defining cube corners in 3D

```

1 vector<Point3f> cubePoints = {
2     {-1, -1, 0}, {1, -1, 0}, {1, 1, 0}, {-1, 1, 0},
3     {-1, -1, -markerLength}, {1, -1, -markerLength},
4     {1, 1, -markerLength}, {-1, 1, -markerLength}
5 };

```

- Projected 3D points to the 2D image using `projectPoints()`.
- Drawn cube edges connecting projected vertices.

Listing 27: Drawing cube on marker

```

1 line(frame, imagePoints[j], imagePoints[(j+1)%4], Scalar(255, 0,
   0), 2);
2 line(frame, imagePoints[j+4], imagePoints[(j+1)%4 + 4], Scalar
   (255, 0, 0), 2);
3 line(frame, imagePoints[j], imagePoints[j+4], Scalar(255, 0, 0),
   2);

```

3.5.3 How to Run

Compilation Same as prev. labs.

Execution Both programs require three arguments:

Listing 28: Run pose estimation

```

1 ./pose_estimation DICT_ARUCO_ORIGINAL camera_calib.yaml 0.043

```

Listing 29: Run cube drawing

```

1 ./draw_cube DICT_ARUCO_ORIGINAL camera_calib.yaml 0.043

```

3.5.4 Results and Discussion

Both programs were able to detect ArUco markers in real time and correctly overlay virtual content. The coordinate system axes helped visualize the marker's 3D pose, and the rendered cube maintained stable alignment with the physical marker. Minor error in the pose was observed due to camera noise or fast motion.

Screenshots of Output:

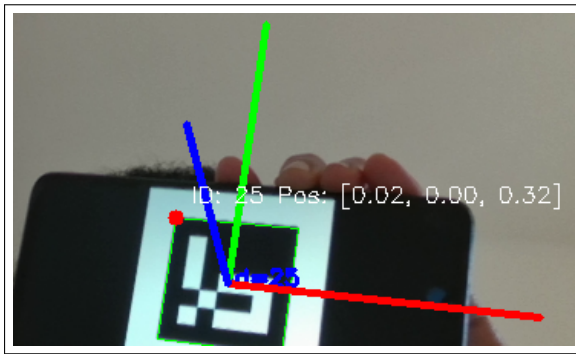


Figure 9: Live pose estimation with axis overlay

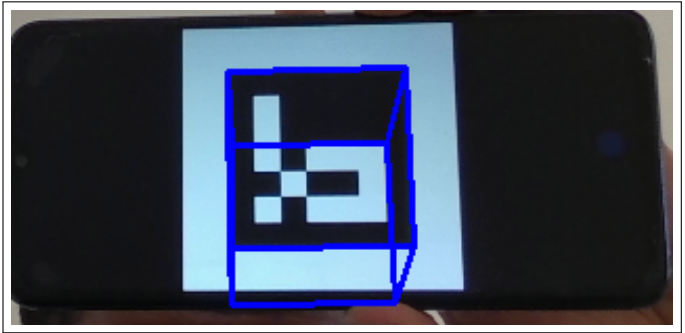


Figure 10: Augmented cube overlay on ArUco marker

4 Conclusion

Across five progressive lab tasks, we successfully developed a complete augmented reality (AR) pipeline using ArUco markers and OpenCV in C++. Each lab built upon the previous one, gradually increasing the complexity and functionality of the system. **Lab 1** laid the foundation for grayscale conversion, Gaussian blur, edge detection (Canny), and binarization techniques. **Lab 2** introduced the generation of fiducial ArUco markers. This lab emphasized the importance of marker dictionaries and visualization quality (white borders for contrast). **Lab 3** focused on real-time marker detection using a webcam. We implemented marker recognition, ID labeling, and visual enhancements like drawing borders and corners. This enabled us to verify the presence and identity of ArUco markers, setting the stage for camera calibration. **Lab 4** we tackled camera calibration using a board of ArUco markers, to compute the camera's intrinsic parameters and distortion coefficients. The resulting `.yaml` file enabled accurate pose estimation in subsequent tasks, achieving a reprojection error of approximately 4.26. **Lab 5** completed the AR pipeline. we first visualized marker pose using 3D axes (`pose_estimation`) and then rendered a virtual cube on top of the marker (`draw_cube`).

Overall, this lab series provided a comprehensive, hands-on understanding of marker-based augmented reality. From image processing and marker generation to calibration and 3D rendering. These applications demonstrated fundamental AR principles—tracking and projection—in real-time using computer vision techniques.

References

- [1] OpenCV Documentation: <https://docs.opencv.org>